Práctica 2: Ingesta y calibración y procesamiento de imágenes con ROS 2

Contenido

1. Objetivo de la Práctica 2

2. Entorno de Trabajo (Punto de Partida) 2

a. ¿Cómo arrancar el entorno? 2

3. Tareas 3

Parte 1: Requisitos Mínimos 3

Parte 2: Retos Opcionales 3

Parte 3: Calidad del Código 3

5. Evaluación y Detalles 5

6. Criterios de Calificación: 5

7. Penalizaciones: 5

8. Entrega 6

9. Recursos de Apoyo y Notas Extra 6

1. Objetivo de la Práctica

El objetivo de esta práctica es familiarizarse con la extracción y procesamiento de datos en entornos robóticos. Trabajaréis con un entorno simulado de un brazo robótico **UR3e** utilizando **ROS 2 Jazzy**.

El objetivo de la práctica es desarrollar un paquete en ROS 2 Jazzy que permita ingerir imágenes a través de un topic de ROS 2 (utilizando el paquete “usb_cam”), calibrar la cámara y procesar las imágenes capturadas mediante OpenCV. Aunque el entorno incluye la simulación del brazo robótico UR3e, para el desarrollo de esta práctica dicho robot no es relevante.

2. Entorno de Trabajo (Punto de Partida)

Partiendo del entorno Dockerizado de ROS2 y **Visual Studio Code**. En el entorno se incluye:

- La simulación completa del robot UR3e.
- Todos los paquetes de ROS 2 necesarios.

Para realizar esta práctica necesitaréis utilizar adicionalmente los paquetes de ROS 2 **“usb_cam” y “camera_calibration”**, así como la librería OpenCV. Además, deberéis generar un paquete propio en ROS 2 Jazzy donde alojaréis todo el contenido desarrollado en la práctica.

a. ¿Cómo arrancar el entorno?

En el archivo "README.md" del repositorio base encontraréis los detalles y requisitos del entorno. Una vez cumplidos requisitos mínimos basta con ejecutar el script proporcionado según vuestro sistema operativo:

- Ejecutar "./lab.sh iniciar" para **Linux** o "lab.bat iniciar" para **Windows** (incluir "--lab" o "--lab-gpu" si se quiere lanzar una de estas opciones)

3. Tareas

Parte 1: Requisitos Mínimos

Para superar la práctica, debéis desarrollar un script en Python que cumpla exactamente con lo siguiente:

1. **Publicador de imágenes:** Crear un publicador que muestre las imágenes de la cámara utilizando el paquete “usb_cam”.
2. **Suscriptor e ingesta de imágenes:** Utilizar un suscriptor que ingiera 10 imágenes seguidas, asegurándoos de que estas sean diferentes entre sí, y las almacene.
3. **Calibración de la cámara:** Calibrar la cámara mediante el paquete **“camera_calibration”** de ROS 2.
4. **Aplicación de la calibración:** Aplicar los parámetros de calibración obtenidos sobre la imagen de la cámara mediante OpenCV. Para ello, debéis convertir el dato tipo **“Image”** de ROS 2 Jazzy a una imagen de OpenCV utilizando obligatoriamente el paquete `cv\_bridge` (no se admiten otros métodos de conversión).

Parte 2: Retos Opcionales

Una vez conseguido el objetivo principal, podéis implementar estas características para optar a una mayor calificación:

- **Detección de ArUco:** Con la cámara ya calibrada, localizar un marcador ArUco en la imagen mediante la función propia de OpenCV.
- **Conversión de píxeles a milímetros:** Con la cámara calibrada y el marcador ArUco detectado, convertir las dimensiones del ArUco de píxeles a milímetros. Sin mover la cámara y moviendo únicamente el ArUco por la escena, comprobad y responded a la siguiente cuestión: ¿se mantienen las relaciones de conversión a lo largo de toda la imagen?

Parte 3: Calidad del Código

Se evaluará que el código entregado sea propio de un entorno "profesional". Se exige:

- Código limpio y bien estructurado. Programación orientada a objetos.
- Uso de comentarios explicativos (no triviales) y Docstrings en funciones y clases.
- Uso de Type Annotations / Type Hinting (indicación de los tipos de variables esperados).

- Seguir el estándar de nomenclatura de Python (PEP-8):

- `variables\_y\_funciones` en minúsculas y separadas por barra baja (mi_función, variable_a).
- `NombresDeClases` con la primera letra en mayúscula y sin espacios ni guiones (MiClase).
- "Funciones privadas de la clase" con barra baja delante del nombre (_function)
- El nombre de variables, clases y funciones debe ser lógico, y entendible, sin variables ni valores mágicos.

5. Evaluación y Detalles

Esta es una **práctica en parejas**, pero **la evaluación del trabajo será estrictamente individual** basándose en la actividad del repositorio.

1. Sobre el uso de GitHub:

El código debe alojarse obligatoriamente en un repositorio de **GitHub**.

- **El trabajo de cada miembro del grupo será evaluado ÚNICAMENTE a través de sus *commits*.** Si un alumno no tiene *commits* a su nombre en el repositorio, su nota será un 0, independientemente del trabajo de su pareja. Debéis trabajar de forma colaborativa usando Git.
- No puede haber ningún contribuidor externo al grupo en el repositorio.

6. Criterios de Calificación:

- **50%:** Cumplimiento de los requisitos mínimos.
- **40%:** Implementación de las funcionalidades opcionales.
- **10%:** Limpieza de código y buenas prácticas.

7. Penalizaciones:

- **Fuera de plazo:** Una entrega fuera de plazo supone una nota máxima del 75% si se entrega en un plazo menor a 24h posteriores de la finalización del plazo original. Cualquier *commit* registrado en el repositorio con una fecha y hora posterior a la fecha límite de entrega se considerará como una entrega tardía, en caso de entrega fuera del plazo de 24 horas extra, la tarea será evaluada como un suspenso.
- **Uso abusivo de IA:** El uso de herramientas de Inteligencia Artificial de forma abusiva o copiando bloques de código sin comprender su funcionamiento supondrá un 0 inmediato. El profesor podrá preguntar al alumno en caso de duda.

8. Entrega

Qué entregar: Únicamente el enlace (URL) a vuestro repositorio de GitHub.

- Aseguraos de que el repositorio sea accesible para el equipo docente el día de la entrega. El nombre del repositorio es libre.

9. Recursos de Apoyo y Notas Extra

Para facilitar el desarrollo y fomentar las buenas prácticas de ingeniería, os recomendamos apoyaros en las siguientes herramientas durante la práctica:

- **Herramienta rqt_graph:** Se trata de una interfaz visual de ROS 2 que os permitirá ver un esquema gráfico de los nodos que están activos y cómo se comunican entre sí a través de los topics. Es una herramienta para depurar errores, entender el flujo de los datos y aseguraros de que vuestro suscriptor está correctamente conectado al topic del robot. Podéis ejecutarla mediante el paquete de ROS 2 "rqt_graph".
- **Documentación Oficial de ROS 2:** Acostumbraos a consultar la documentación y los tutoriales oficiales de ROS 2 (específicamente para la distribución Jazzy). Es la fuente más fiable para entender el funcionamiento de los suscriptores, la creación de mensajes personalizados y la estructura de los paquetes. Encima está bien escrita ;).

Sabemos que ROS 2 y Docker pueden tener una curva de aprendizaje inicial compleja. Si os encontráis con un error que os impide avanzar y habéis intentado solucionarlo sin éxito apoyándoos tanto en la documentación oficial como mediante preguntas a la IA, **no dudéis en preguntar al profesor**. Estamos aquí para ayudaros a entender los conceptos y guiaros en la resolución de problemas. No esperéis al último día para preguntar, es posible que no podamos responder en plazos tan cortos de tiempo y no se harán excepciones.